Drawing Fair Congressional Districts Using Computer Algorithms

A Guide for Everyone

January 2026

Abstract

Every 10 years after the census, states must redraw congressional district boundaries. This process, called redistricting, is often manipulated by politicians to favor their party—a practice known as gerrymandering. This document explains how we can use computer algorithms to draw district maps fairly and transparently, without human bias. We'll walk through the concepts using real examples from Minnesota and Alabama, showing how a technique called *edge-weighted recursive bisection* can automatically create compact, fair districts.

Contents

1 The Problem: Drawing 435 Districts **1**

2 Redistricting as a Graph Problem **2**

2.1 Graph Partitioning for Redistricting 3

2.1.1 Compactness 4

2.2 Partitioning with Edge Weights 5

2.3 Unequal Splits 6

2.4 Recursive Bisection: Splitting Repeatedly 7

3 Real Examples: Minnesota and Alabama **8**

3.1 Minnesota: 8 Districts 8

3.2 Alabama: 7 Districts 9

3.3 Why These Examples Matter 10

4 National Results: All 50 States **10**

5 Why This Matters **12**

5.1 Advantages of Algorithmic Redistricting 12

5.2 Limitations and Trade-offs 12
5.3 The Path Forward 13

6 Conclusion **13**

A More Examples: Different Cities and Population Ratios **15**
A.1 Minneapolis, MN: 50-50 Split (1:1) 15
A.2 Houston, TX: 60-40 Split (3:2) 15
A.3 Los Angeles, CA: 43-57 Split (3:4) 17
A.4 Atlanta, GA: 45-55 Split (5:6) 17
A.5 Phoenix, AZ: 46-54 Split (6:7) 18
A.6 Miami, FL: 47-53 Split (8:9) 18
A.7 Summary of Examples 19

1 The Problem: Drawing 435 Districts

The House of Representatives has 435 members, each representing one district. Every 10 years, the Census determines **apportionment**—how many districts each state gets based on population. After 2020: California has 52, Texas 38, Minnesota 8, Alabama 7, Alaska 1. Once apportioned, someone must draw the boundaries. This is where things get tricky.

Gerrymandering manipulates boundaries to favor one party via **packing** (concentrate opposition voters to waste votes) or **cracking** (spread them thin to prevent majorities). Named for Massachusetts Governor Elbridge Gerry's 1812 salamander-shaped district, gerrymandering produces bizarre, non-compact shapes. These manipulated districts often look like abstract art—long tendrils reaching across counties, strange appendages connecting distant communities, or jigsaw-puzzle pieces designed to capture specific neighborhoods while excluding others.

The stakes are high: gerrymandering can determine which party controls Congress, even when that party receives fewer total votes. It reduces electoral competition, creating "safe seats" where outcomes are predetermined. Voters in gerrymandered districts often feel their votes don't matter, contributing to political polarization and decreased trust in democratic institutions.

The Constitution requires **equal population** (760,000 in 2020) and **contiguity** (connected regions). Beyond this, good districts should be **compact** (not bizarre shapes), **fair** (not favoring one party), and **respect communities** (not unnecessarily splitting cities). Currently, most states leave redistricting to state legislatures—the same politicians who benefit from favorable district lines. This creates an inherent conflict of interest.

Our goal: Can computers draw compact, fair districts automatically, removing human bias from the process?

2 Redistricting as a Graph Problem

Redistricting is a complex spatial problem. With most programming problems, the first step is to solve the base case. In this case, that would be to split a region into two equal-population parts.

Why start with the two-way split? Because it's the simplest redistricting problem that captures all the key challenges: balancing population, maintaining geographic compactness, and ensuring contiguity. Once we master this building block, scaling up becomes straightforward through repetition.

Here's an example of twelve tracts in Minneapolis, Minnesota. Our goal is to split this region into two equal-population parts (50-50 split). These aren't hypothetical shapes—they're real census tract boundaries that follow streets, rivers, and neighborhood boundaries.

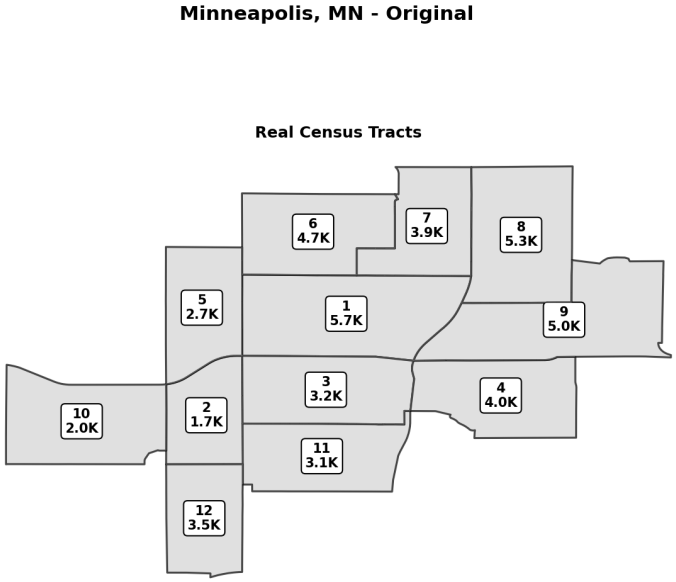


Figure 1: **Real Census Tracts (2010)**: Twelve actual tracts from Hennepin County, Minnesota.

We introduce a methodology to convert this map into an abstract graph representation. Each **census tract** (small area of 1,200-8,000 people; 85,000 nationwide) becomes a vertex sized by population. Adjacent tracts connect via edges.

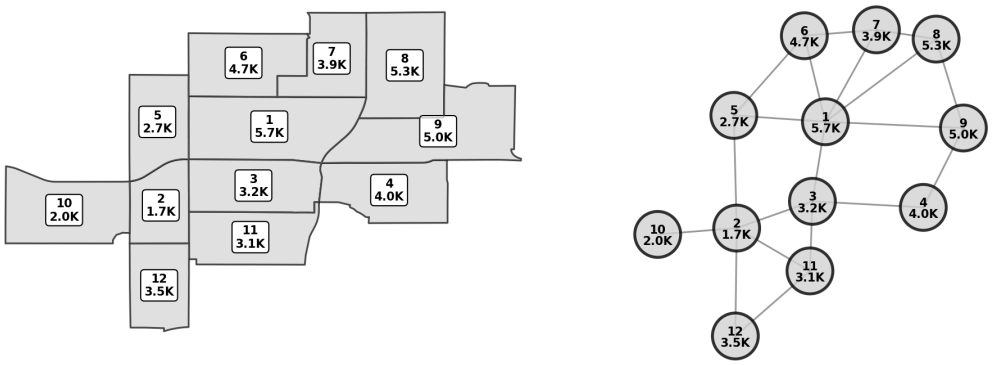


Figure 2: **Tracts to Graph**: Left: The same twelve tracts. Right: The abstract graph.

This representation preserves geography (adjacent tracts remain connected), ensures contiguity automatically (any partition must form connected components), and lets us apply powerful algorithms from other fields like engineering and scientific computing.

2.1 Graph Partitioning for Redistricting

Redistricting becomes a graph partitioning problem: split into k connected groups with equal population. This is not a new problem—engineers have been partitioning graphs for decades to distribute computational work across processors, minimize communication costs in computer networks, and optimize circuit layouts. We're applying the same mathematical tools to a different domain.

METIS is a graph partitioning library from the University of Minnesota, widely used in engineering and scientific computing. Given a graph, target populations, and connectivity requirements, METIS quickly returns two balanced, connected groups. It uses sophisticated multilevel algorithms that are both fast (handling graphs with millions of nodes) and effective (finding high-quality partitions). METIS has been refined over decades and tested in countless applications, making it a battle-tested tool for our redistricting needs.

For our example in Minnesota, we run METIS on the graph and ask it to split the graph into 2 components. METIS reads a specially-formatted text file describing the graph structure, then outputs the partition assignment.

Format explanation:

- **Header line**: `12 19 001` means 12 vertices (tracts), 19 edges, format code 001 (vertex sizes: yes, edge weights: no)
- **Data lines**: Population followed by neighbor IDs
- **Output file**: One partition ID (0 or 1) per tract

METIS Input (format 001):

```
12 19 001
5732 3 5 6 7 8 9
1728 3 5 10 11 12
3229 1 2 4 11
4022 3 9
2719 1 2 6
4732 1 5 7
3916 1 6 8
5308 1 7 9
4972 1 4 8
1978 2
3115 2 3 12
3532 2 11
```

METIS Output:

```
0
1
1
1
0
0
0
0
1
1
1
1
```

Result: Tracts 1,5,6,7,8 → partition 0 (blue); Tracts 2,3,4,9,10,11,12 → partition 1 (red)

Here's what that looks like geographically:

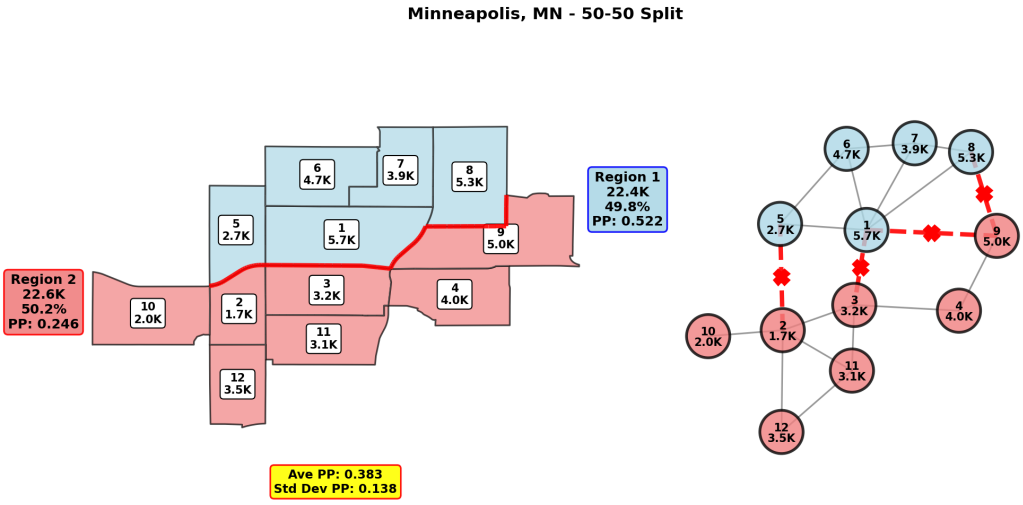


Figure 3: **Graph Partitioning**: METIS creates two separate graphs of equal population.

If you compare closely, you can see that the boundaries that are cut in the map are equal to the edges that are cut in the graph.

2.1.1 Compactness

We measure compactness using the **Polsby-Popper score**: $\frac{4\pi \times \text{Area}}{\text{Perimeter}^2}$. A perfect circle scores 1.0, a square scores 0.79, and gerrymandered districts score <0.2. Our goal is to generate districts with scores of 0.25 to 0.30 and higher. Why does this formula work? It rewards shapes with large areas relative to their perimeters. Circles and squares are compact; snakes and salamanders are not. The formula provides an objective, numerical measure of the "bizarre shape" problem we're trying to avoid.

In this example, region 1 is quite compact at 0.522 but region 2 is 0.246. The average PP is 0.383 but the standard deviation is 0.138.

2.2 Partitioning with Edge Weights

Edge-weighted partitioning assigns each edge a weight equal to its actual boundary length. Only after meeting population targets does it minimize boundary length. This simple addition transforms the algorithm from counting edges to measuring real geography. We compute edge weights by loading census tract geometries, projecting to meters, finding where boundaries intersect, and measuring the shared length.[1] A long boundary between two tracts is more "costly" than a short boundary. By making METIS aware of these costs, we guide it toward geographic compactness.

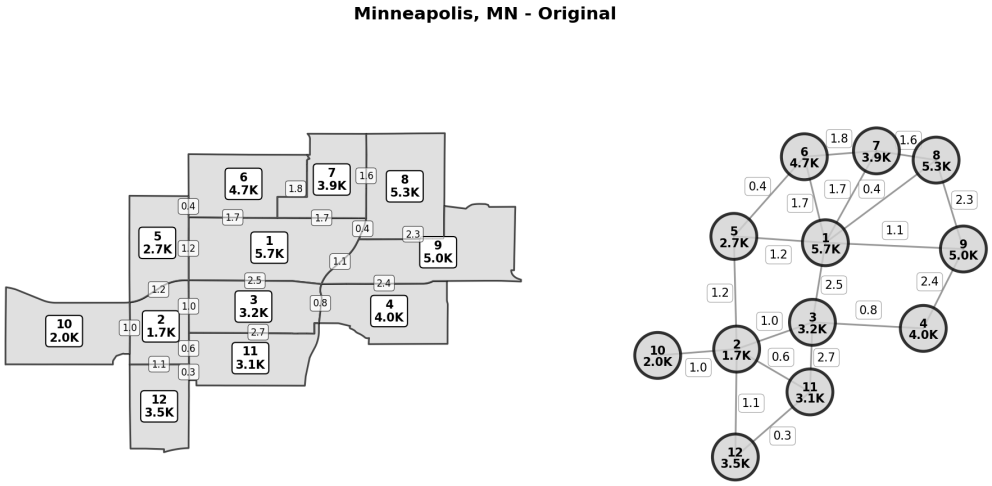


Figure 4: **Edge Weights**: The boundary lengths (km) become edge weights in the graph.

Now we pass this geographic data to METIS using the edge-weighted format (011), again showing input and output side-by-side:

Format change:

- **Header**: `12 19 011` uses format 011 (vertex sizes: yes, edge weights: yes)
- **Data lines**: Now alternate `population neighbor_id edge_weight ...` format
- **Edge weights**: Measured in meters (as integers, which METIS requires)

METIS Input (format 011):

```
12 19 011
5732 3 2539 5 1200 6 1693 7 1695 8 426 9 1125
1728 3 1003 5 1224 10 981 11 600 12 1133
3229 1 2539 2 1003 4 754 11 2683
4022 3 754 9 2400
2719 1 1200 2 1224 6 401
4732 1 1693 5 401 7 1808
3916 1 1695 6 1808 8 1609
5308 1 426 7 1609 9 2255
4972 1 1125 4 2400 8 2255
1978 2 981
3115 2 600 3 2683 12 301
3532 2 1133 11 301
```

METIS Output:

```
1
1
1
0
1
0
0
0
0
1
1
1
```

[1] Water crossings use artificial "bridge" edges with reasonable weights matching typical mainland boundaries within the same county.

Result: Tracts 4,6,7,8,9 → partition 0 (blue); Tracts 1,2,3,5,10,11,12 → partition 1 (red)

Notice the difference: tracts are assigned differently than the unweighted version. The populations are still balanced (49-51 split), but METIS chose different boundaries to minimize total perimeter length. This new assignment creates a more compact geographic split:

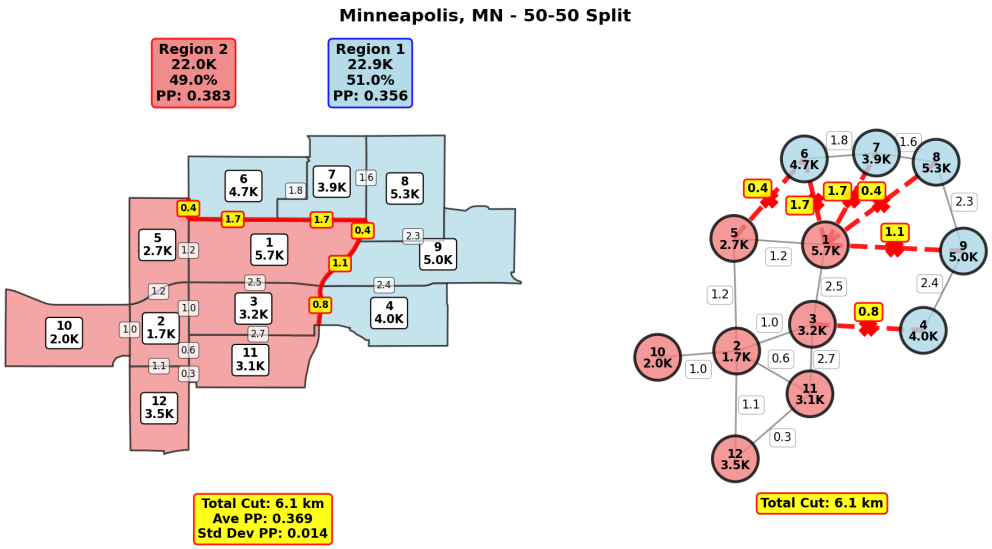


Figure 5: **Edge Weights in Action**: METIS is able to cut a shorter boundary using edge weights.

METIS seeks to minimize the total edge weight of the cut, which in our case equals the total boundary length between the two resulting districts. By minimizing boundary length, we consistently create compact districts!

Both region 1 and region 2 are quite compact - at 0.356 and 0.383 respectively, forming an average of 0.369. While this is a bit lower than the previous average of 0.383, the standard deviation is lower by an order of magnitude at 0.014.

2.3 Unequal Splits

Edge weights work equally well for unequal population splits. When a state needs an odd number of districts, the algorithm must create **unequal splits** (like 2:1 or 3:2). These unequal partitions arise naturally during recursive bisection—for example, splitting 7 districts requires a 4-3 split in the first round, and splitting 3 further requires a 2-1 split. Edge weights ensure these unequal partitions still follow natural geographic boundaries while respecting the required population ratios.

The algorithm handles these seamlessly by simply telling METIS the target population ratio via a weights file:

```
0 = 0.667
1 = 0.333
```

This 2-line file specifies partition 0 gets 67% and partition 1 gets 33% (a 2:1 ratio). METIS uses the same graph structure as before but produces a different partition to meet this unequal target

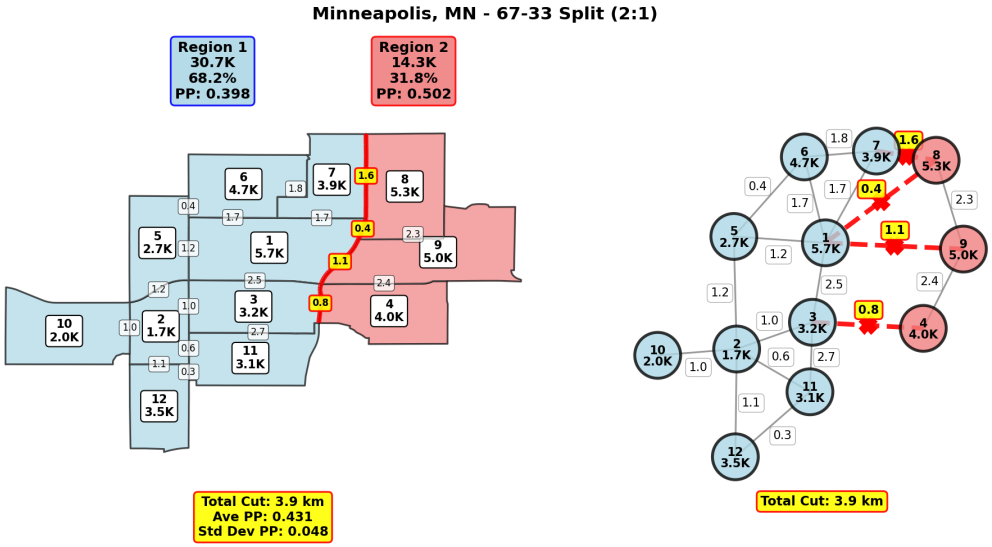


Figure 6: **Unequal Splits**: The same twelve Minneapolis tracts split 2:1 (67-33) instead of 50-50.

The algorithm successfully creates a 67-33 population split while minimizing boundary length. As a result, region 1 has a PP of 0.398 and region 2 has a PP of 0.502, with an average of 0.431, a standard deviaton of 0.048, and an even shorter cut than the equal split.

2.4 Recursive Bisection: Splitting Repeatedly

Now that we understand graph partitioning, how do we use it? **Recursive bisection** repeatedly splits regions in half. Start with the whole state, bisect into two equal-population parts, then recursively split each part until reaching the target district count. This approach is elegant because it breaks a complex problem (creating 8 or 38 or 52 districts) into many simple problems (repeatedly creating 2 districts).

Why is recursion natural here? Think about how you might manually draw districts. You'd probably start by splitting the state into large regions, then subdivide those regions, continuing until you reach individual districts. Recursive bisection formalizes this intuitive approach, applying the same two-way split operation at each step until the job is complete. The algorithm is simple enough to implement in 50 lines of code, yet powerful enough to handle any state.

Recursive Bisection Algorithm - RBA$(\mathcal{R}, P, n)$:
Input: $\mathcal{R}$: region, P: population, n: number of districts
Output: $\{(\mathcal{R}_1, P_1, 1), \ldots, (\mathcal{R}_n, P_n, 1)\}$ where:

- $\mathcal{R}_1 \cup \cdots \cup \mathcal{R}_n = \mathcal{R}$ (partition covers entire region)
- $\mathcal{R}_i \cap \mathcal{R}_j = \emptyset$ for $i \neq j$ (regions are disjoint)
- $P_1 + \cdots + P_n = P$ (population conservation)
- $P_1, \ldots, P_n \approx P/n$ (equal population per district)

Algorithm:

1. **Base case:** If $n = 1$, return $\{(\mathcal{R}, P, 1)\}$
2. **Recursive case:** Calculate how to split n into two parts:
   - If n is **even**: $n_1 = n/2$, $n_2 = n/2$ (equal split)
   - If n is **odd**: $n_1 = \lceil n/2 \rceil$, $n_2 = \lfloor n/2 \rfloor$ (adjacent integers)
3. **Partition:** $(\mathcal{R}, P, n) \to (\mathcal{R}_1, P_1, n_1)$ and $(\mathcal{R}_2, P_2, n_2)$ where:
   - $\mathcal{R}_1 \cup \mathcal{R}_2 = \mathcal{R}$ (partition covers entire region)
   - $\mathcal{R}_1 \cap \mathcal{R}_2 = \emptyset$ (regions are disjoint)
   - $P_1 + P_2 = P$ (population conservation)
   - $n_1 + n_2 = n$ (district conservation)
   - $P_1/n_1 \approx P_2/n_2 \approx P/n$ (equal population per district)
4. **Recurse:** Return $\text{RBA}(\mathcal{R}_1, P_1, n_1) \cup \text{RBA}(\mathcal{R}_2, P_2, n_2)$

Compactness: Polsby-Popper score $\text{PP}(\mathcal{R}) = \frac{4\pi \times \text{Area}(\mathcal{R})}{\text{Perimeter}(\mathcal{R})^2}$
Goals:

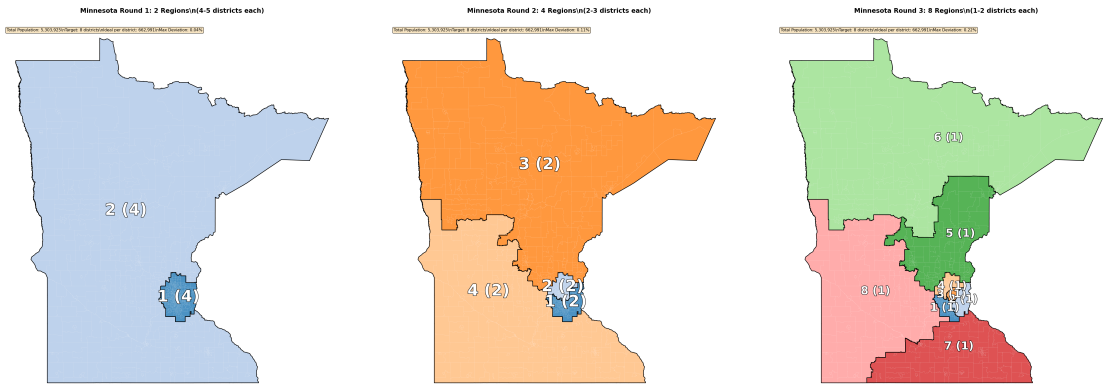
- Maximize average PP: $\max \frac{1}{n} \sum_{i=1}^{n} \text{PP}(\mathcal{R}_i)$
- Minimize PP standard deviation: $\min \sqrt{\frac{1}{n} \sum_{i=1}^{n} (\text{PP}(\mathcal{R}_i) - \overline{\text{PP}})^2}$

3 Real Examples: Minnesota and Alabama

Let's see how this works on real states. Minnesota and Alabama provide an excellent contrast: Minnesota has 8 districts (a power of 2, requiring only equal splits), while Alabama has 7 districts (requiring unequal splits). Together, they demonstrate the algorithm's versatility across different scenarios.

3.1 Minnesota: 8 Districts

Minnesota's 8 districts ($8 = 2^3$) demonstrate perfect recursive bisection through three equal-split rounds.



Round 1: 1→2
$8 \to 4 + 4$

Round 2: 2→4
$4 \to 2 + 2$

Round 3: 4→8
$2 \to 1 + 1$

Figure 7: Three rounds create Minnesota's 8 districts.

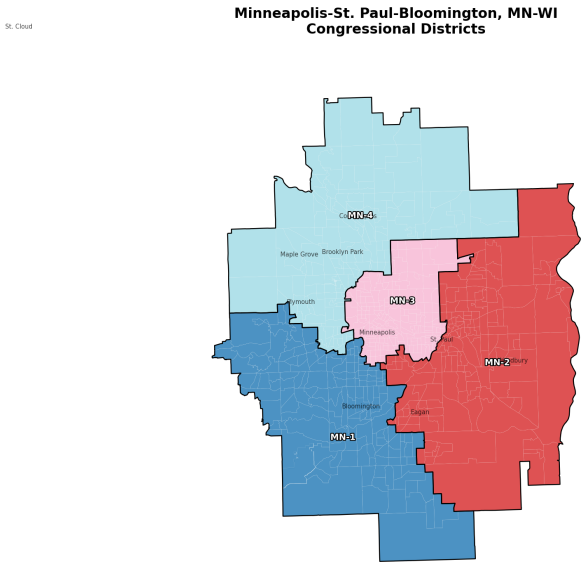


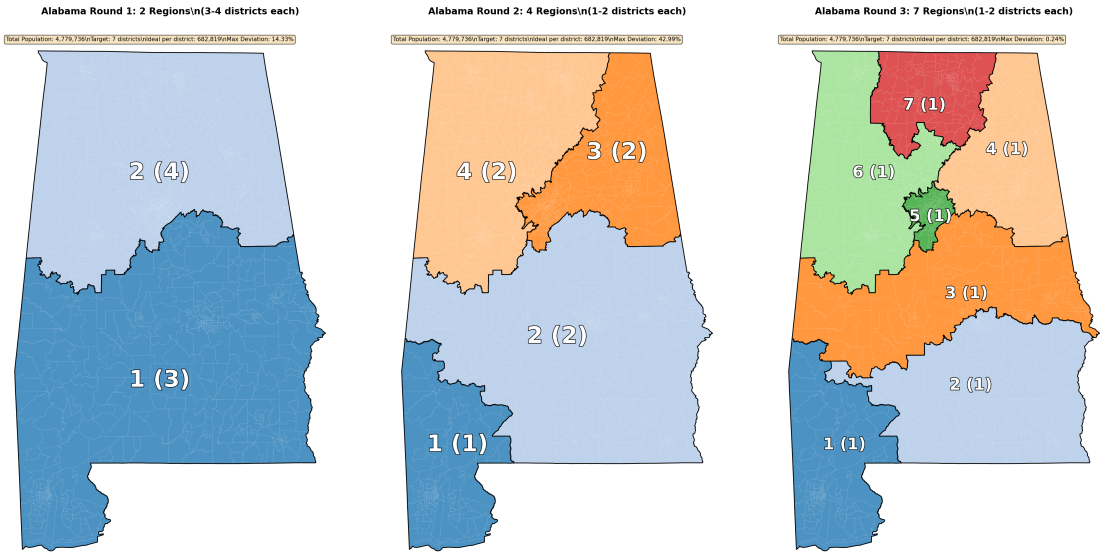
Figure 8: Minneapolis metro area: 4 urban districts with compact boundaries.

Results: Mean Polsby-Popper 0.468—beating both baseline (+62%) and enacted maps (+63%). This means our algorithm creates districts that are significantly more compact than either randomly-generated districts or the districts actually drawn by the Minnesota legislature. The improvement is substantial enough to be visually obvious.

3.2 Alabama: 7 Districts

Alabama's 7 districts demonstrate adaptive splitting for odd numbers. Since 7 isn't a power of 2, the algorithm uses unequal splits (4+3, then 2+2+2+1) while maintaining compactness. This is more representative of typical states—most have district counts that aren't powers of 2. Notice how the algorithm handles this gracefully: the first round splits 7→4+3 (as balanced as possible),

then subsequent rounds handle these regions independently.



Round 1: 1→2
$7 \to 4 + 3$

Round 2: 2→4
$4 \to 2 + 2,\ 3 \to 2 + 1$

Round 3: 4→7
$2 \to 1 + 1$

Figure 9: Three rounds create Alabama's 7 districts.

Results: Mean Polsby-Popper 0.369—beating both baseline (+50%) and enacted maps (+24%). The +24% improvement over enacted districts is particularly notable, suggesting Alabama's actual 2020 districts may contain some gerrymandering. Our algorithm provides an objective baseline for comparison.

3.3 Why These Examples Matter

Minnesota and Alabama demonstrate that edge-weighted recursive bisection works well for both:

- **Even district counts** (powers of 2): Perfect balanced splitting with symmetric tree structure
- **Odd district counts** (non-powers of 2): Algorithm adapts automatically using adjacent integers
- **Different geographies**: Minnesota's lakes vs Alabama's varied terrain, urban vs rural
- **Different political contexts**: The algorithm performs well regardless of the state's partisan lean

4 National Results: All 50 States

Testing on all 435 congressional districts shows edge-weighted bisection achieves mean Polsby-Popper 0.367—a 56% improvement over unweighted baseline (0.235) and 20% better than enacted 2020 maps (0.305). We beat enacted maps in 37 of 50 states. These numbers represent real improvements in compactness that would be visible to anyone looking at the maps: districts that follow more natural boundaries, fewer bizarre shapes, and more geographic coherence.

The 56% improvement over unweighted baseline demonstrates the value of edge weights—simply encoding real boundary lengths makes a dramatic difference. The 20% improvement over enacted districts suggests that many states' actual redistricting included either intentional gerrymandering or simply suboptimal geographic decisions.

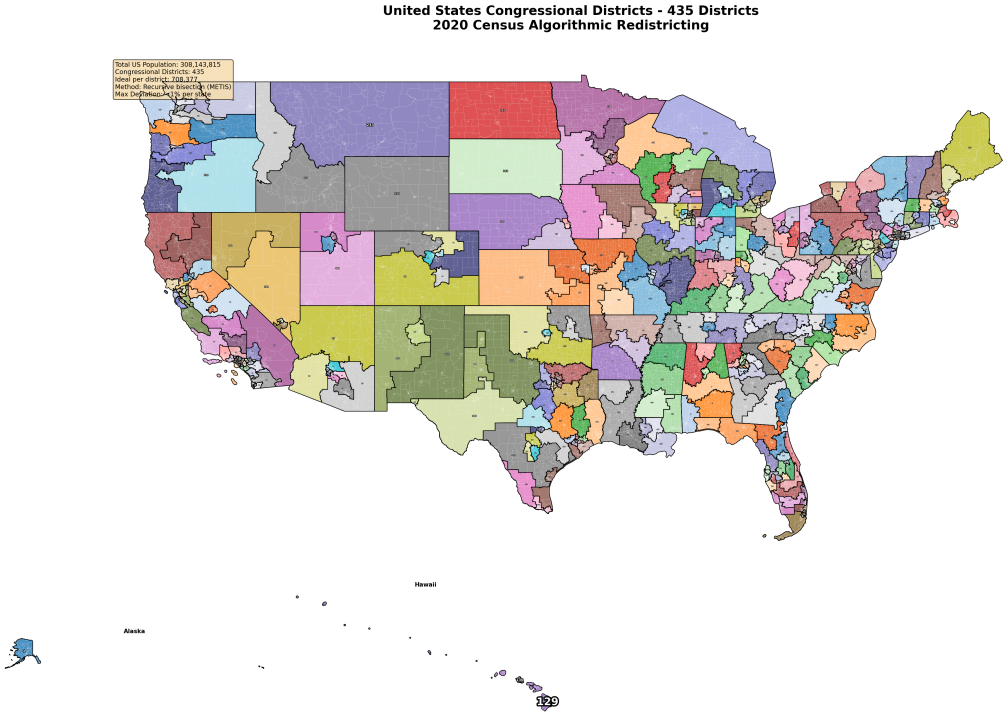


Figure 10: **All 435 Congressional Districts (2020)**: The complete national map shows our algorithmically-generated districts for all 50 states. Each district is colored uniquely, with Alaska and Hawaii shown as insets. The algorithm automatically created compact districts across diverse geographic contexts—from dense urban areas to sprawling rural regions. No manual intervention was required—the same code ran on all states.

Biggest improvements occur in suspected gerrymandered states: Illinois (+174%), Louisiana (+104%), New Hampshire (+102%), Maryland (+99%), and North Carolina (+76%). These massive improvements—some states more than doubling their compactness scores—provide objective evidence of gerrymandering. When an algorithm produces districts that are 174% more compact than human-drawn districts, something has gone wrong in the human process.

Cross-census validation: Testing on 2010 data yields similar compactness (0.320 vs 0.367), proving **geography drives our algorithm, not politics**. This is crucial: if our algorithm were somehow encoding partisan preferences, we'd expect different results when applied to different census years with different political contexts. The consistency across time periods confirms that we're optimizing for geography, not outcomes.

5 Why This Matters

5.1 Advantages of Algorithmic Redistricting

1. Eliminates partisan manipulation: No human control over lines, removing politicians' incentive to gerrymander. The algorithm doesn't know which party lives where—it only knows populations and boundary lengths. This makes partisan manipulation mathematically impossible. Politicians can't adjust lines to benefit their party when they're not drawing the lines.

2. Complete transparency: Open-source code anyone can run and verify—no "smoke-filled room" decisions. Every step of the process is documented and reproducible. Anyone with a computer can download the code, run it themselves, and verify they get identical results. This level of transparency is impossible with human-drawn districts, where backroom negotiations and political considerations remain hidden.

3. Provides a neutral baseline: Creates objective standards for detecting gerrymandering by comparing algorithmic to human-drawn maps. When human-drawn districts are dramatically less compact than algorithmic ones (like Illinois's 174% difference), we have quantitative evidence of manipulation. The algorithm provides the "expected" result if geography alone determined boundaries.

4. Speed and efficiency: Runs in minutes, enabling rapid scenario testing without legislative deadlock. All 50 states can be redistricted in under an hour on a standard computer. This speed allows testing multiple scenarios (different census data, different population tolerances) without the months of legislative wrangling that typically accompanies redistricting.

5.2 Limitations and Trade-offs

1. No consideration of communities: Pure geography-based—may split cities or counties that could stay together. The algorithm optimizes for compact boundaries without considering whether a boundary splits a school district, divides a neighborhood, or separates a community of interest. Human judgment might preserve these communities even at the cost of slightly less compact districts.

2. No attempt at competitive districts: Optimizes compactness, not electoral competition—may create safe seats. Some reformers prefer "competitive" districts where elections could go either way, believing this increases accountability. Our algorithm makes no such consideration—it creates compact districts whether they end up politically competitive or not.

3. Doesn't ensure minority representation: May not naturally create Voting Rights Act-compliant districts without added constraints. The Voting Rights Act requires some districts to have majority-minority populations to ensure representation. Our algorithm doesn't consider race or ethnicity, so it might not naturally create these districts. Additional constraints would be needed to ensure VRA compliance.

4. Still requires human judgment: Data granularity, population tolerance, and special cases need decisions. Should we use census tracts or census blocks? Allow $\pm 0.5\%$ population deviation or require exact equality? How should island communities be handled? These decisions, while less politically charged than drawing districts, still require human input.

5.3 The Path Forward

Algorithmic redistricting provides: (1) a baseline for detecting gerrymandering, (2) a starting point for human adjustments, (3) a neutral default when politicians deadlock, and (4) public education on fair districts. The goal is removing partisan abuse and establishing objective standards, not replacing all human judgment.

Think of it like spell-check: the algorithm provides a strong default that handles most cases correctly, but humans can override when special circumstances warrant. The key difference from current practice: any deviation from the algorithmic baseline must be explicitly justified, making gerrymandering much harder to hide. Courts could require states to explain why their districts differ significantly from the algorithmic version, placing the burden of proof on those seeking to deviate from geographic optimality.

6 Conclusion

By representing states as graphs with edge weights equal to real boundary lengths, we automatically create compact districts that often surpass human-drawn maps. Our approach combines three key ideas: recursive bisection (break complex problems into simple two-way splits), METIS graph partitioning (leverage proven algorithms from computer science), and edge weights (encode real geography into the optimization). This combination is powerful yet simple enough to explain to non-experts.

Results: 56% improvement over unweighted baseline, 20% improvement over enacted 2020 maps, with particularly strong performance in gerrymandered states and consistent results across census years. These aren't just numbers—they represent districts that look more natural, follow geographic boundaries better, and avoid the bizarre shapes that signal manipulation. The fact that we beat enacted maps in 37 of 50 states suggests that geographic optimization often conflicts with human decision-making, whether that's intentional gerrymandering or just suboptimal choices.

Impact: Algorithmic redistricting provides objectivity, transparency, and neutral baselines for measuring fairness—a significant step toward reducing gerrymandering and restoring public trust. Perhaps most importantly, it changes the conversation: instead of arguing about whether a particular district "looks" gerrymandered, we can quantitatively measure how much it deviates from the algorithmic baseline and demand justification for that deviation.

The path to fairer districts doesn't require perfect algorithms or eliminating all human judgment—it requires changing the default. Make the algorithmic baseline the starting point, allow human adjustments when justified, but require transparency and explanation for deviations. This shifts power from politicians drawing favorable districts to a process that starts with geographic optimality and only deviates when there's a legitimate reason.

For More Information

Want to learn more or try it yourself?

- **Full technical paper**: See `papers/03_combined_recursive_bisection/`
- **Source code**: Available on GitHub
- **Interactive maps**: Visit our web dashboard to explore districts for all 50 states

- **Questions?** Contact the author

Key references:

- METIS graph partitioning: `http://glaros.dtc.umn.edu/gkhome/metis/metis/overview`
- U.S. Census Bureau redistricting data: `https://www.census.gov/programs-surveys/decennial-census/about/rdo.html`
- Polsby-Popper compactness measure: Standard metric in redistricting research

A More Examples: Different Cities and Population Ratios

In the main text, we showed how our algorithm works with a 50-50 population split. But real congressional districts often need different population ratios—especially in states with odd numbers of districts or varying population densities.

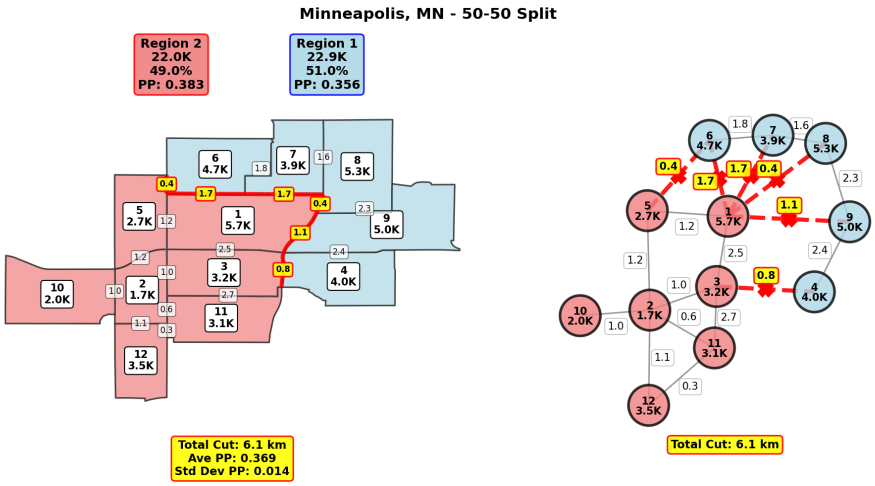
Below are six examples showing how the algorithm handles different population splits across major U.S. metropolitan areas. Each example uses 12 real census tracts and shows:

- **Left panel**: Geographic census tracts colored by partition
- **Right panel**: Abstract graph representation with edge weights
- **Red lines/edges**: METIS-selected cuts that minimize total boundary length
- **Yellow labels**: Boundary lengths (in kilometers)
- **Blue/Red regions**: Two partitions with population totals

Note on example selection: These examples were specifically chosen to demonstrate both accurate population ratios (within 0.5% of target) and good compactness (Polsby-Popper score $\geq$ 0.25 for both regions). To find these particularly clean examples with only 12 tracts, we tested multiple starting locations in each metropolitan area and selected the best result. In practice, state-level redistricting works with hundreds or thousands of census tracts, which provides much more flexibility for the algorithm to achieve precise population targets while maintaining compact, contiguous districts. These small examples are designed for pedagogical clarity, not to represent the full complexity of real redistricting.

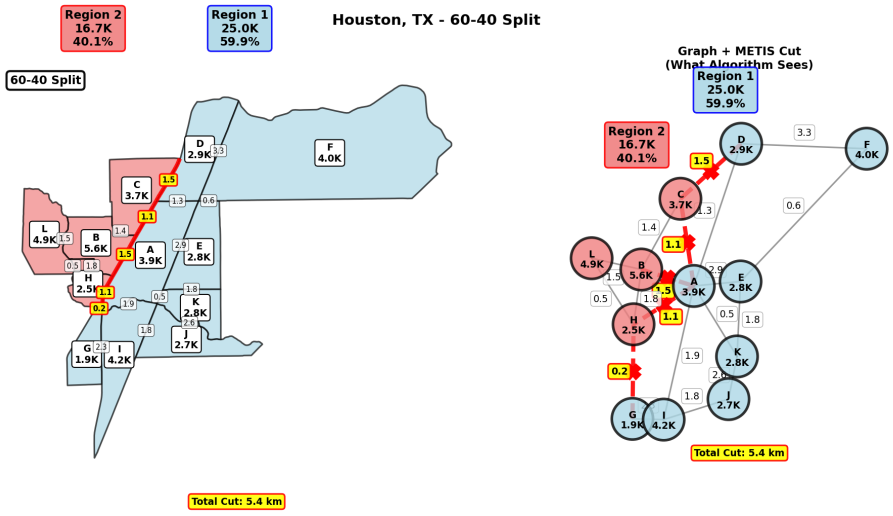
A.1 Minneapolis, MN: 50-50 Split (1:1)

This is the example from the main text. Equal population split (1:1 ratio).



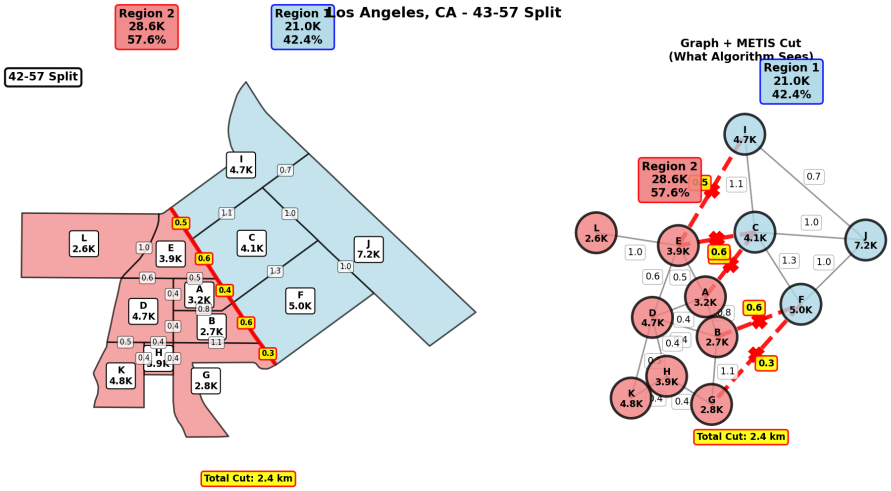
A.2 Houston, TX: 60-40 Split (3:2)

A 3:2 ratio shows moderate population difference. The algorithm finds compact boundaries despite the imbalance.


Houston, TX - 60-40 Split
Region 2
16.7K
40.1%
Region 1
25.0K
59.9%
60-40 Split
D
2.9K
F
4.0K
C
3.7K
L
4.9K
B
5.6K
A
3.9K
E
2.8K
H
2.5K
K
2.8K
J
2.7K
G
1.9K
I
4.2K
Total Cut: 5.4 km
Graph + METIS Cut
(What Algorithm Sees)
Region 1
25.0K
59.9%
Region 2
16.7K
40.1%
Total Cut: 5.4 km

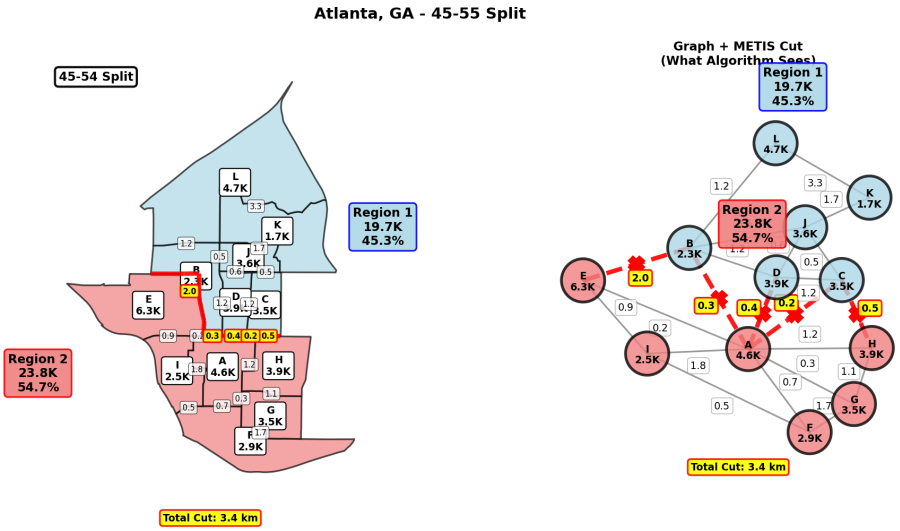
A.3 Los Angeles, CA: 43-57 Split (3:4)

A 3:4 ratio demonstrates a slight majority. Notice how METIS adjusts the partition to meet the target population.



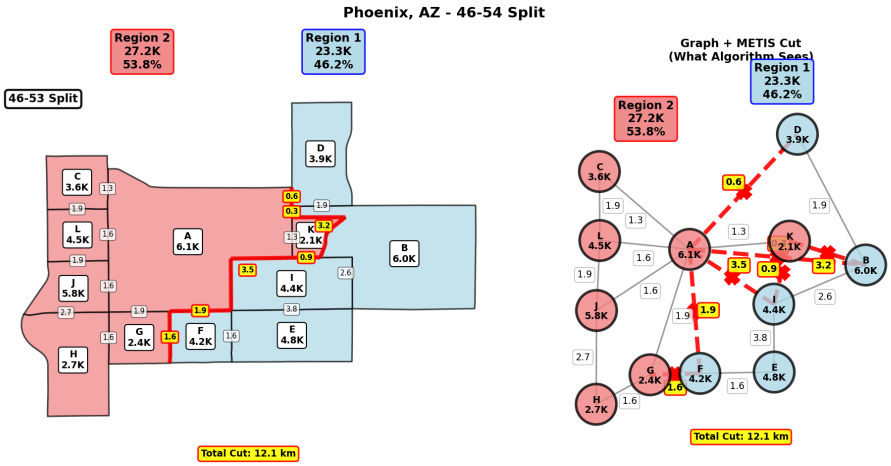
A.4 Atlanta, GA: 45-55 Split (5:6)

A 5:6 ratio is close to equal but not quite. This shows the algorithm's precision in meeting exact population targets.



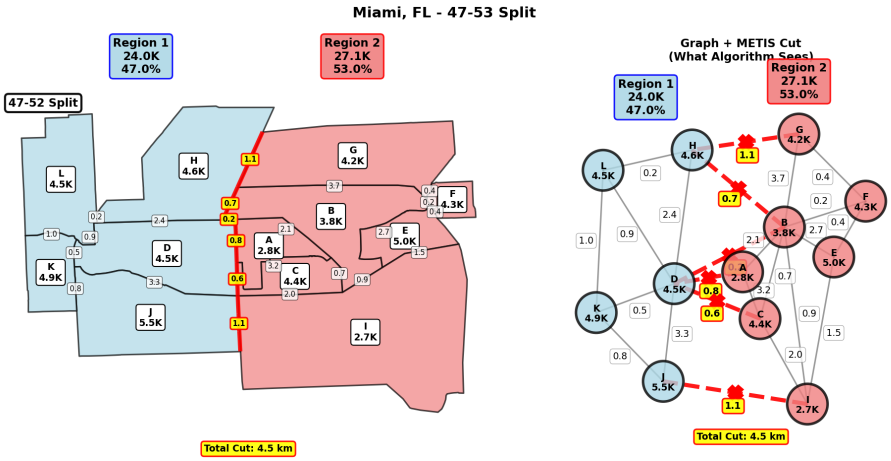
A.5 Phoenix, AZ: 46-54 Split (6:7)

A 6:7 ratio is nearly balanced. Even with this close split, the algorithm finds an optimal cut.



A.6 Miami, FL: 47-53 Split (8:9)

An 8:9 ratio is very close to equal. This demonstrates the algorithm's ability to handle nearly-equal splits.



A.7 Summary of Examples

These six examples demonstrate several important points:

1. **Flexibility**: The algorithm works with any population ratio, from equal (50-50) to close (47-53) to moderate (60-40).
2. **Consistency**: Whether the split is balanced or imbalanced, METIS always minimizes boundary length.
3. **Real geography matters**: Edge weights based on actual boundary lengths produce geographically sensible districts.
4. **Automatic adaptation**: The algorithm adjusts the cut location to meet population targets while staying compact.
5. **Contiguity preserved**: All splits maintain connected regions, even with challenging population ratios.

In real redistricting, states don't always divide evenly. Texas needs 38 districts, Florida needs 28, and many states have odd counts. Our algorithm handles all of these cases automatically, finding the best cut for whatever population ratio is needed at each step of the recursive bisection process.

The power of this approach: No manual tweaking required. Specify the population target, and METIS finds the most compact way to achieve it.

Scale matters: It's important to note that these appendix examples use only 12 census tracts for visual clarity. Real state-level redistricting works with far more data:

- California has over 9,000 census tracts
- Texas has over 5,000 census tracts
- Even small states like Delaware have 200+ tracts

With hundreds or thousands of tracts available, the algorithm has much more flexibility to find solutions that simultaneously meet strict population targets, maintain high compactness, and preserve natural geographic boundaries. The curated examples shown here demonstrate the *principles* of edge-weighted bisection in an easily understandable format, but they should not be interpreted as representing the full capability or typical performance of the algorithm when applied to complete state redistricting.